

Practical 7

```
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC, SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score, confusion_matrix
import pathlib as pl
import os

## DisableWarning: sklearn.exceptions.ConvergenceWarning
import warnings
from sklearn.exceptions import ConvergenceWarning
warnings.simplefilter(action='ignore', category=ConvergenceWarning)

script_dir = pl.Path(__file__).parent.absolute()
os.chdir(script_dir)

df = pd.read_csv('iris.csv')
print(df.head())

encoder = LabelEncoder()
df.Species = encoder.fit_transform(df.Species)

train, test = train_test_split(df, test_size=0.2, random_state=35)

train_x = train.drop(columns=['Species'], axis=1)
train_y = train['Species']

test_x = test.drop(columns=['Species'], axis=1)
test_y = test['Species']

models = {
    "Logistic Regression": LogisticRegression(),
    "SVM (one-vs-one)": SVC(),
    "Linear SVM (one-vs-rest)": LinearSVC(dual=False),
    "Decision Tree (Criterion : Entropy)": DecisionTreeClassifier(criterion='entropy')
}

for name, model in models.items():
    print(f"Model: {name}")
    model.fit(train_x, train_y)
    predict = model.predict(test_x)
    # actual_vs_predict = [f"{i} | {j}" for i, j in
zip(encoder.inverse_transform(test_y), encoder.inverse_transform(predict))]
    # print('Actual vs Predicted')
    # for i in actual_vs_predict:
    #     print(i)
    print(f"Accuracy: {accuracy_score(test_y, predict)}")
    print(f"Confusion Matrix: \n{confusion_matrix(test_y, predict)}")
```

File Edit Selection View Go Run ...

EXPLORER

- PS
- code
- iris.csv
- ps6.py
- ps7.py
- ps7.pdf
- P10_P10PS_merged... U
- PS.docx
- PS6.pdf
- Sem6_PSPRACTICAL_4_to_1...

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

COMMENTS

```

/Users/Aditya/Desktop/sem6/SUBJECTS/PS/code/ps7.py
  Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm  Species
0  1      5.1         3.5         1.4         0.2  Iris-setosa
1  2      4.9         3.0         1.4         0.2  Iris-setosa
2  3      4.7         3.2         1.3         0.2  Iris-setosa
3  4      4.6         3.1         1.5         0.2  Iris-setosa
4  5      5.0         3.6         1.4         0.2  Iris-setosa

Model: Logistic Regression
Accuracy: 1.0
Confusion Matrix:
[[11  0  0]
 [ 0 10  0]
 [ 0  0  9]]

Model: SVM (one-vs-one)
Accuracy: 1.0
Confusion Matrix:
[[11  0  0]
 [ 0 10  0]
 [ 0  0  9]]

Model: Linear SVM (one-vs-rest)
Accuracy: 1.0
Confusion Matrix:
[[11  0  0]
 [ 0 10  0]
 [ 0  0  9]]

Model: Decision Tree (Criterion : Entropy)
Accuracy: 1.0
Confusion Matrix:
[[11  0  0]
 [ 0 10  0]
 [ 0  0  9]]

(base) PS C:\Users\Aditya\Desktop\sem6\SUBJECTS\PS>

```

OUTLINE

main* 0 0

Ln 50, Col 69 Spaces: 4 UTF-8 CRLF {} Python 3.12.9 (weny) Go Live

1014 20-03-2025

Practical 9

```
import pathlib as pl
import os
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.model_selection import cross_val_score

script_dir = pl.Path(__file__).parent.absolute()
os.chdir(script_dir)

df = pd.read_csv('iris.csv')

feature_columns = ['SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm']
X = df[feature_columns].values
y = df['Species'].values

encoder = LabelEncoder()
y = encoder.fit_transform(y)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=35)

sns.pairplot(df.drop("Id", axis=1), hue='Species', height=2, markers=['o', 's', 'D'])

classifier = KNeighborsClassifier(n_neighbors=3)
classifier.fit(X_train, y_train)

y_pred = classifier.predict(X_test)

cm = confusion_matrix(y_test, y_pred)
print(f"Confusion Matrix: \n{cm}")

accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy}")

plt.show()
```

